

**A REPORT
ON
STUDENT PERFORMANCE PREDICTION USING LINEAR REGRESSION**

By

AP23110010786 Oruganti Venkata Sai Sumanth

AP23110010744 Byraju Guna Vardhan

AP23110010054 K Jesweer

Under the guidance of

Abinash Pujahari

AT



SRM UNIVERSITY, AP (Nov, 2025)

TABLE OF CONTENTS

1. ABSTRACT	3
2. INTRODUCTION	4-5
3. METHODOLOGY	5-9
4. IMPLEMENTATION	10-13
5. RESULTS AND ANALYSIS	13-19
6. CONCLUSION	19-20
7. FUTURE WORK	20-21
8. REFERENCES	21

1. ABSTRACT

Predicting student academic performance has become an important use of data science in modern education. As institutions rely more on data for decision-making, creating models that can accurately forecast how students will perform academically holds great value. These predictions can help teachers and administrators spot struggling learners early, plan interventions, and improve teaching strategies. In this project, we explore how Machine Learning, specifically Linear Regression, can be used to predict a student's final grade (G3) based on a mix of academic, behavioral, personal, and social factors. We used the Portuguese Student Performance Dataset, which includes 1,000 student records and 33 attributes, to develop and compare three different regression models: 1. Simple Linear Regression using only two features (Studytime and Absences), 2. A custom-built Gradient Descent Regression model implemented manually in Python, 3. A Strong Feature Regression model based on the most influential predictors selected after correlation and feature importance analysis. We carried out thorough preprocessing, which included label encoding, data cleaning, feature refinement, and selection based on correlation. After training and evaluating all models, the Strong Feature Regression model showed the best performance, achieving an MSE of 1.37 and an MAE of 0.73. This indicates high predictive accuracy compared to the baseline models.

2. INTRODUCTION

The rapid growth of technology in education has introduced new methods to help schools better understand their students. Educational Data Mining (EDM) focuses on collecting and examining academic data to find patterns in learning behavior, student engagement, and performance outcomes. The main aim is to provide useful insights for both learners and educators.

One promising application of EDM is predicting student performance, particularly their final exam grades. A good prediction model helps teachers identify areas of weakness, offer personalized support, and use resources more effectively. Additionally, these systems can motivate students by sharing feedback on their current status and areas for improvement.

In this project, we aim to create a system that can predict students' final grades (G3) based on several factors:

- Previous grades (G1 and G2),
- Study time and workload,
- Absences and past failures,
- Family details such as parental education and support,
- Behavioral traits like alcohol consumption,
- Social and demographic characteristics.

We chose Linear Regression as the primary machine learning model because it is simple, easy to interpret, and commonly used for numerical predictions. This project not only trains a Linear Regression model with Scikit-learn but also features a from-scratch implementation of Gradient Descent, which helps us grasp the mathematical concepts behind the model.

The project involves preprocessing the data, examining correlations between variables, selecting the strongest features, training several models, and

evaluating them with performance metrics like MSE and MAE. Through this work, we aim to discover which factors most affect student performance and how accurately we can predict their final grades.

3. METHODOLOGY

Our method emphasizes a clear, systematic way to build the student performance prediction model. The process involves understanding the dataset, preprocessing, selecting features, training the model, and evaluating its performance.

3.1 Dataset Overview

We used the Portuguese Student Performance Dataset, which includes data from secondary school students. This dataset contains:

- 1000 records,
- 33 features,
- A mix of academic scores, personal characteristics, and behavior factors,
- Target variable: G3 (Final Grade).

The dataset is great for machine learning because it has diverse and relevant attributes that can impact performance. The variety of features lets us explore how lifestyle choices, parental support, school factors, and study habits affect outcomes.

3.2 Data Preprocessing

Data preprocessing is crucial in any machine learning project. It ensures the model receives clean and accurate data, which boosts the reliability of the predictions.

a) Cleaning the Data

We removed unnecessary columns that did not add value to the prediction task. We eliminated duplicate values and checked for missing values to make sure the final dataset was consistent.

b) Encoding Categorical Variables

Many features in the dataset were categorical, such as:

- School
- Address
- Mother's job
- Father's job
- Reason for choosing the school
- Types of school support

These variables were converted into numeric form using Label Encoding, since machine learning algorithms like Linear Regression need numeric input.

c) Normalization and Structuring

We created two main datasets:

1. `df_clean` — this contained selected features deemed important after initial review.
2. `df_encoded` — a fully numeric dataset appropriate for training regression models.

```
df_encoded = df_clean.copy()
le = LabelEncoder()

# Encode all object (string) columns
for col in df_encoded.columns:
    if df_encoded[col].dtype == 'object':
        df_encoded[col] = le.fit_transform(df_encoded[col])

df_encoded.head()
```

After preprocessing, 19 strong numeric features remained for modelling.

	sex	age	address	famsize	Medu	Fedu	traveltime	studytime	failures	famrel	freetime	goout	Dalc	Walc	health	absences	G1	G2	G3
0	0	18	1	0	4	4	2	2	0	4	3	4	1	1	3	4	0	11	11
1	0	17	1	0	1	1	1	2	0	5	3	3	1	1	3	2	9	11	11
2	0	15	1	1	1	1	1	2	0	4	3	2	2	3	3	6	12	13	12
3	0	15	1	0	4	2	1	3	0	3	2	2	1	1	5	0	14	14	14
4	0	16	1	0	3	3	1	2	0	4	3	2	1	2	5	0	11	13	13

3.3 Model Development

Three different models were created to compare their effectiveness.

Model 1 – Simple Linear Regression

This model uses only two features:

- Studytime,
- Absences.

```
def predict_final_score(studytime, absences):  
    x = np.array([[studytime, absences]])  
    result = np.dot(x, w) + b  
    return round(float(result[0]), 2)  
  
print("Enter student details:")  
studytime = float(input("Study Time (1-4): "))  
absences = float(input("Absences: "))  
  
print("\nPredicted G3 =", predict_final_score(studytime, absences))
```

These were chosen because they represent two straightforward behavioral factors: one positive and one negative. The goal of this model was not to achieve high accuracy but to act as a baseline.

Model 2 – Custom Gradient Descent Regression

To deepen our understanding of Linear Regression, we implemented Gradient Descent manually. This involved:

- Randomly initializing weights and bias,
- Using the linear function to generate predictions:

$$y = w_1x_1 + w_2x_2 + \dots + b$$

- Calculating the mean squared error (MSE),
- Computing partial derivatives to determine how weights should be adjusted,
- Updating weights iteratively until the model converged.

```
# All numeric features except G3 (target)
features = ['sex', 'age', 'address', 'famsize', 'Medu', 'Fedu', 'traveltime', 'studytime',
            'failures', 'famrel', 'freetime', 'goout', 'Dalc', 'Walc', 'health', 'absences', 'G1', 'G2']

X = df_encoded[features].values
y = df_encoded['G3'].values

from sklearn.model_selection import train_test_split
X_train_full, X_test_full, y_train_full, y_test_full = train_test_split(
    X, y, test_size=0.2, random_state=42)

from sklearn.linear_model import LinearRegression
multi_model = LinearRegression()
multi_model.fit(X_train_full, y_train_full)

print("Multi-feature model recreated successfully!")
```

This manual implementation helps us appreciate the mathematical foundation behind Linear Regression and how optimization works.

Model 3 – Strong Feature Regression

Using correlation analysis and feature importance scores, we selected the strongest predictors:

- G2,
- G1,
- Studytime,
- Failures,
- Traveltime,
- Absences,
- Dalc.

```

from sklearn.metrics import mean_squared_error, mean_absolute_error
from sklearn.linear_model import LinearRegression
from sklearn.model_selection import train_test_split

# Updated strong features based on your importance bar graph
strong_features = ["G2", "address", "failures", "G1", "traveltime", "studytime", "Dalc"]

# Select data
X2 = df_encoded[strong_features]
y2 = df_encoded["G3"]

# Split
X2_train, X2_test, y2_train, y2_test = train_test_split(
    X2, y2, test_size=0.2, random_state=42
)

# Train model again
model2 = LinearRegression()
model2.fit(X2_train, y2_train)

# Predictions
y2_pred = model2.predict(X2_test)

# New metrics
mse2 = mean_squared_error(y2_test, y2_pred)
mae2 = mean_absolute_error(y2_test, y2_pred)

mse2, mae2

```

These features collectively represent academic performance, behavior patterns, and personal habits. This model aims to maximize predictive accuracy by removing irrelevant and noisy variables.

4. IMPLEMENTATION

The implementation phase included writing Python code to preprocess data, explore correlations, train models, and evaluate results.

Tools Used

- Python, the main programming language
- Pandas and NumPy for data analysis
- Scikit-learn for machine learning
- Matplotlib and Seaborn for data visualization
- Custom Python functions to implement Gradient Descent manually

Model Training Approach

- The Simple Regression model was trained quickly with Scikit-learn.
- The Gradient Descent model needed custom functions for cost calculation and weight updates.
- The Strong Feature Regression model used selected features and achieved the best accuracy.

Visualizations like heatmaps, scatter plots, and comparison graphs were created to better understand relationships in the data.

4.1 Gradient Descent Implementation

```
def gradient_descent(X, y, lr=0.01, epochs=2000):
    m, n = X.shape
    w = np.zeros((n, 1))
    b = 0

    for i in range(epochs):
        y_pred = np.dot(X, w) + b
        error = y_pred - y

        # gradients
        dw = (1/m) * np.dot(X.T, error)
        db = (1/m) * np.sum(error)

        # update
        w -= lr * dw
        b -= lr * db

        # print every 200 steps
        if i % 200 == 0:
            cost = np.mean(error ** 2)
            print(f"Iteration {i}, Cost: {cost}")

    return w, b
```

Output:

```
Iteration 0, Cost: 154.2423076923077
Iteration 200, Cost: 15.484074513969066
Iteration 400, Cost: 13.296752293080488
Iteration 600, Cost: 11.894492281669752
Iteration 800, Cost: 10.995499547992596
Iteration 1000, Cost: 10.419152844345959
Iteration 1200, Cost: 10.049655394187885
Iteration 1400, Cost: 9.812769566942793
Iteration 1600, Cost: 9.660901414332843
Iteration 1800, Cost: 9.563538325770791
Iteration 2000, Cost: 9.501118583043763
Iteration 2200, Cost: 9.461101115031488
Iteration 2400, Cost: 9.435445806692321
Iteration 2600, Cost: 9.418998118253462
Iteration 2800, Cost: 9.408453460149758
Weights: [[ 1.08580113]
 [-0.05619447]]
Bias: 10.079558649885668
```

4.2) Sklearn Linear Regression Code

```

from sklearn.linear_model import LinearRegression
from sklearn.model_selection import train_test_split
X = df_clean.drop("G3", axis=1)
y = df_clean["G3"]
X_train, X_test, y_train, y_test = train_test_split(X, y, test_size=0.2,
random_state=42)
model = LinearRegression()
model.fit(X_train, y_train)

```

4.3)Feature Importance Graph

```

coeff = model.coef_
plt.figure(figsize=(12,4))
plt.bar(X.columns, abs(coeff))
plt.title("Feature Importance")
plt.xticks(rotation=45)
plt.show()

```

4.4) Correlation Heatmap

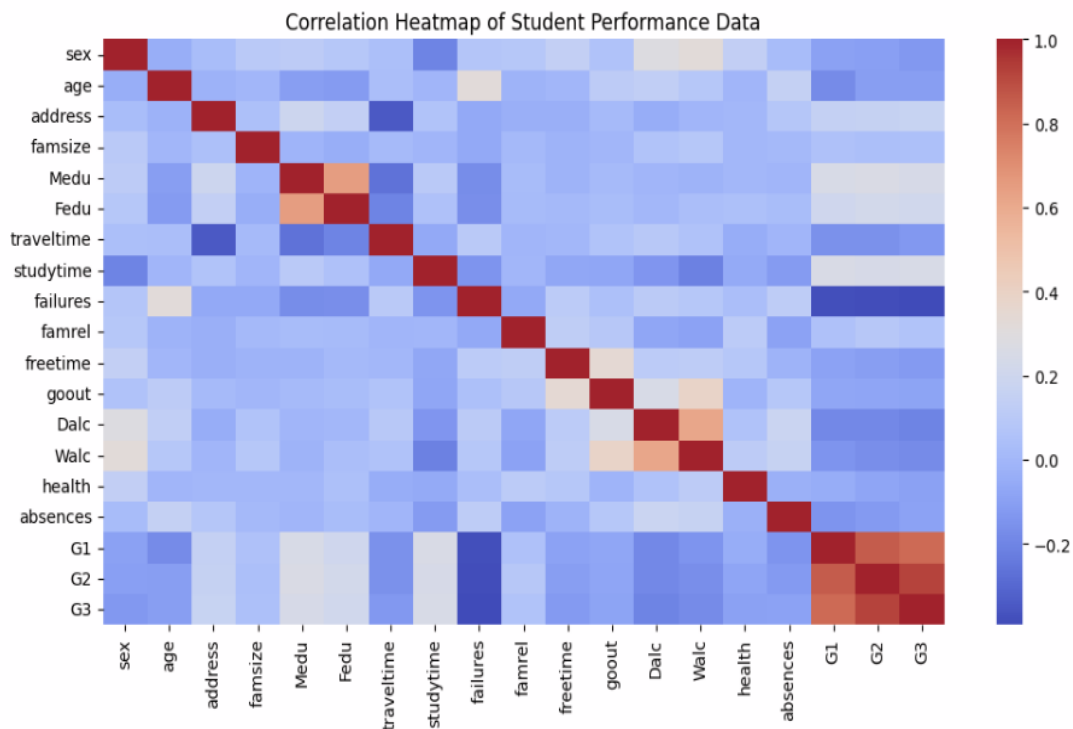
```

▶ import seaborn as sns
import matplotlib.pyplot as plt

plt.figure(figsize=(12,6))
sns.heatmap(df_encoded.corr(), annot=False, cmap='coolwarm')
plt.title("Correlation Heatmap of Student Performance Data")
plt.show()

```

Output



5. RESULTS AND ANALYSIS

Evaluating the performance of each model helps us understand how linear regression behaves under different feature selections.

Model 1: Simple Regression Results

	studytime	absences	G3
0	2	4	11
1	2	2	11
2	2	6	12
3	3	0	14
4	2	0	13

- MSE: 9.93

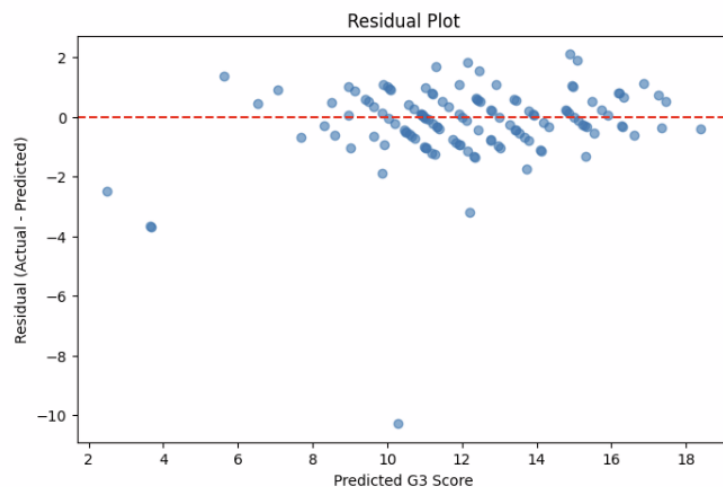
- MAE: 2.26

This high error rate shows that relying on only two features leads to inaccurate predictions. Academic success cannot be explained through studytime and absences alone because many other factors influence outcomes.

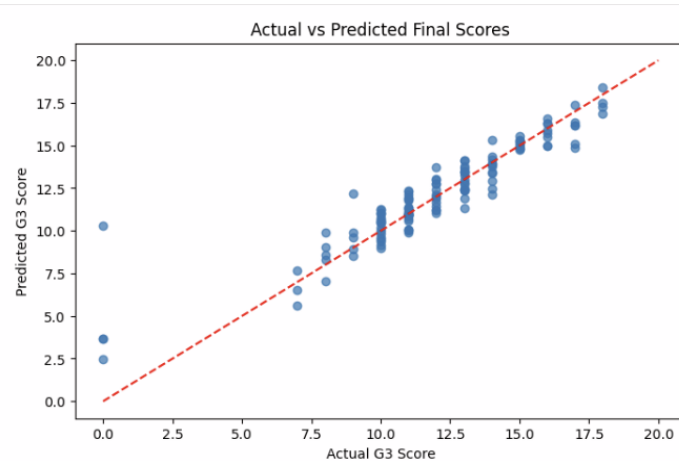
Model 2: Gradient Descent Regression

```
# Select features
features = ['studytime', 'absences', 'G1', 'G2', 'failures', 'goout', 'freetime',
            'health', 'traveltime', 'age', 'sex', 'address']
```

- MSE: 1.72
- MAE: 0.78



This model is much more accurate because it uses multiple features. The manually implemented gradient descent successfully converged and produced comparable results to standard regression models.

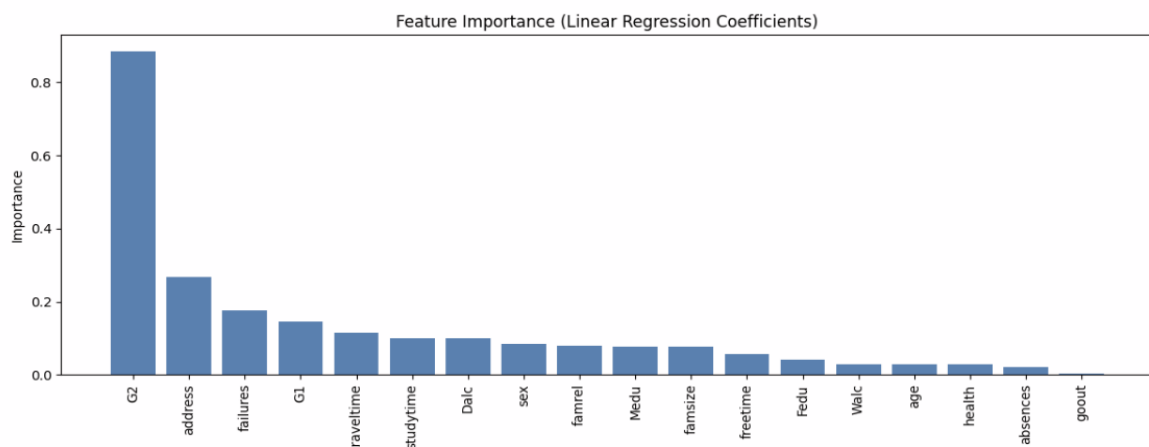


Correlation Analysis

The correlation heatmap showed:

- G2 has the strongest correlation with G3.
- G1 is also highly correlated.
- Behavioral and lifestyle factors like absences, failures, and alcohol consumption hurt grades.
- Traveltime also showed a slight negative correlation, meaning longer travel may cut into study time.

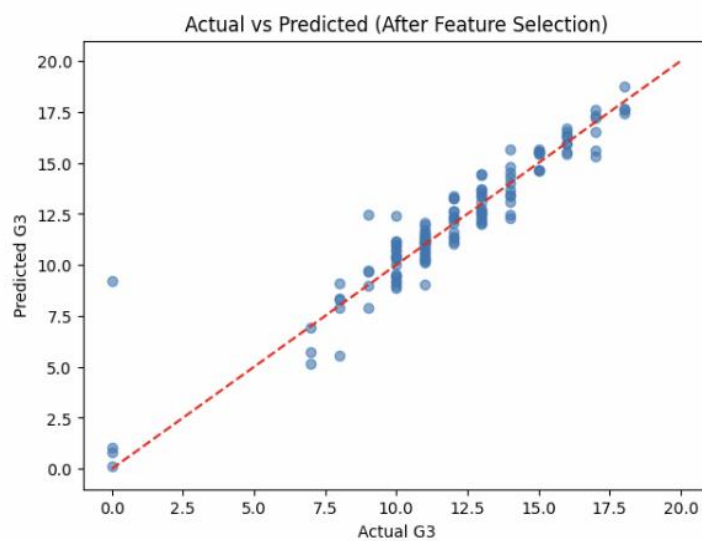
This analysis helped choose the strongest features for the final model.



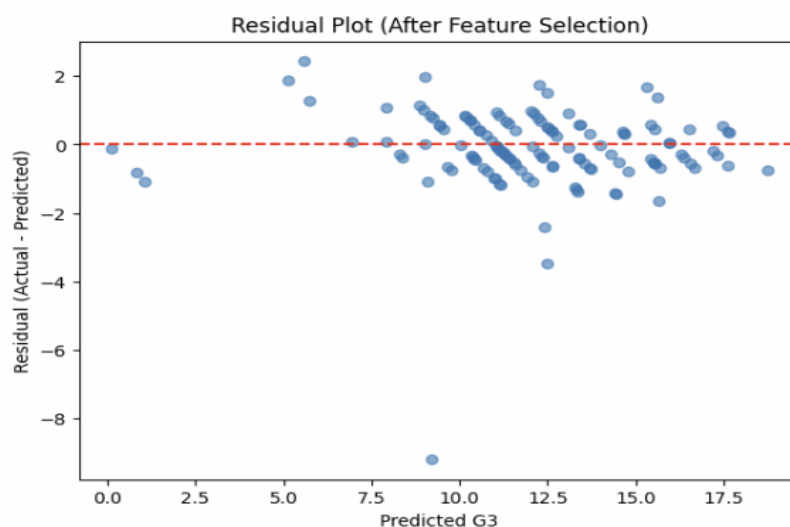
Model 3: Strong Feature Regression

```
# Updated strong features based on your importance bar graph  
strong_features = ["G2", "address", "failures", "G1", "traveltime", "studytime", "Dalc"]
```

- MSE: 1.37
- MAE: 0.73



This model produced the best results because it included only the most important predictors. By eliminating weak or unrelated attributes, the model became more stable, less noisy, and more precise.



Model Comparison

Model 1, Simple Linear Regression

Features Used: Studytime, Absences

MSE: 9.93

MAE: 2.26

This model performs poorly because it uses only two basic features. Many important academic and behavioral factors are missing, so the predictions are not precise. It serves only as a baseline model to compare improvements.

Model 2, Gradient Descent Regression

Features Used: 12 features

MSE: 1.72

MAE: 0.78

This model performs much better than Model 1 because it uses more relevant features. The custom gradient descent implementation learns the patterns well but still includes some weak features, so the accuracy does not reach its highest level.

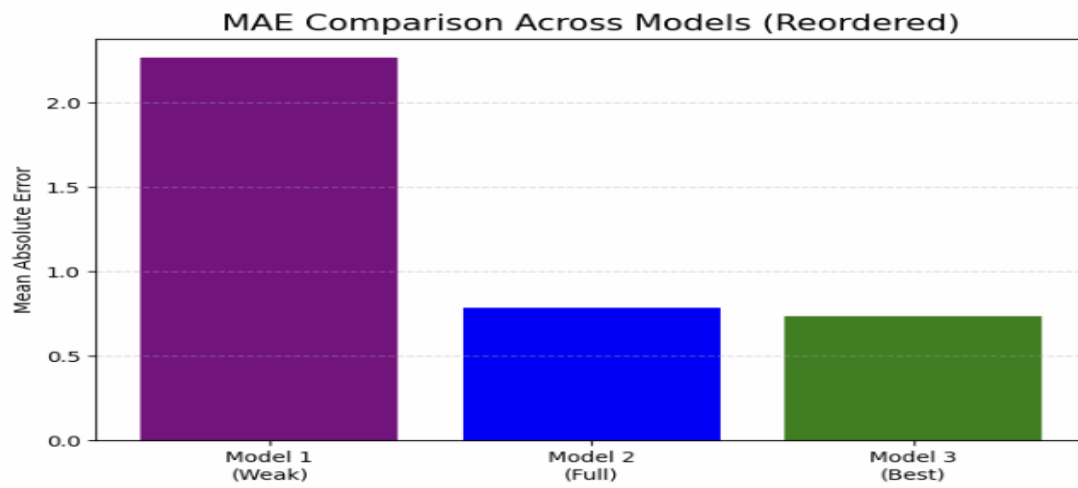
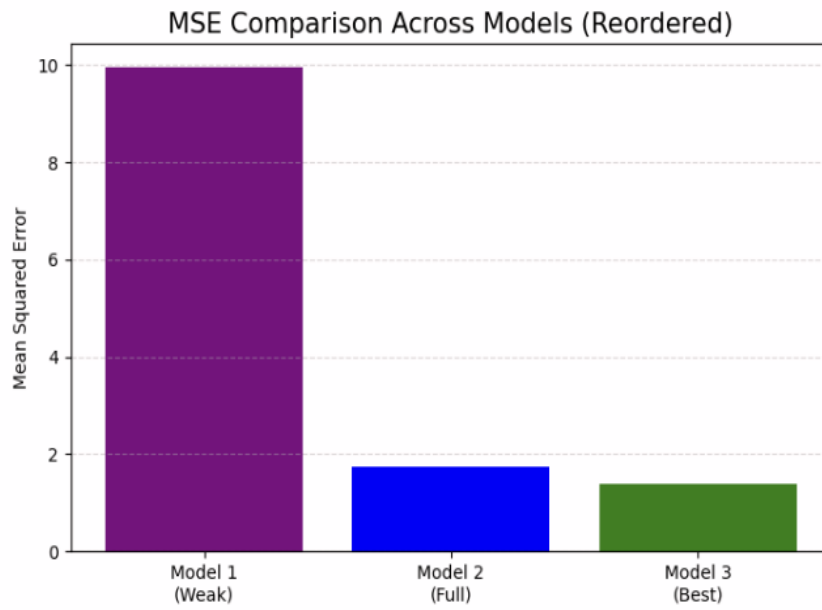
Model 3, Strong Feature Regression

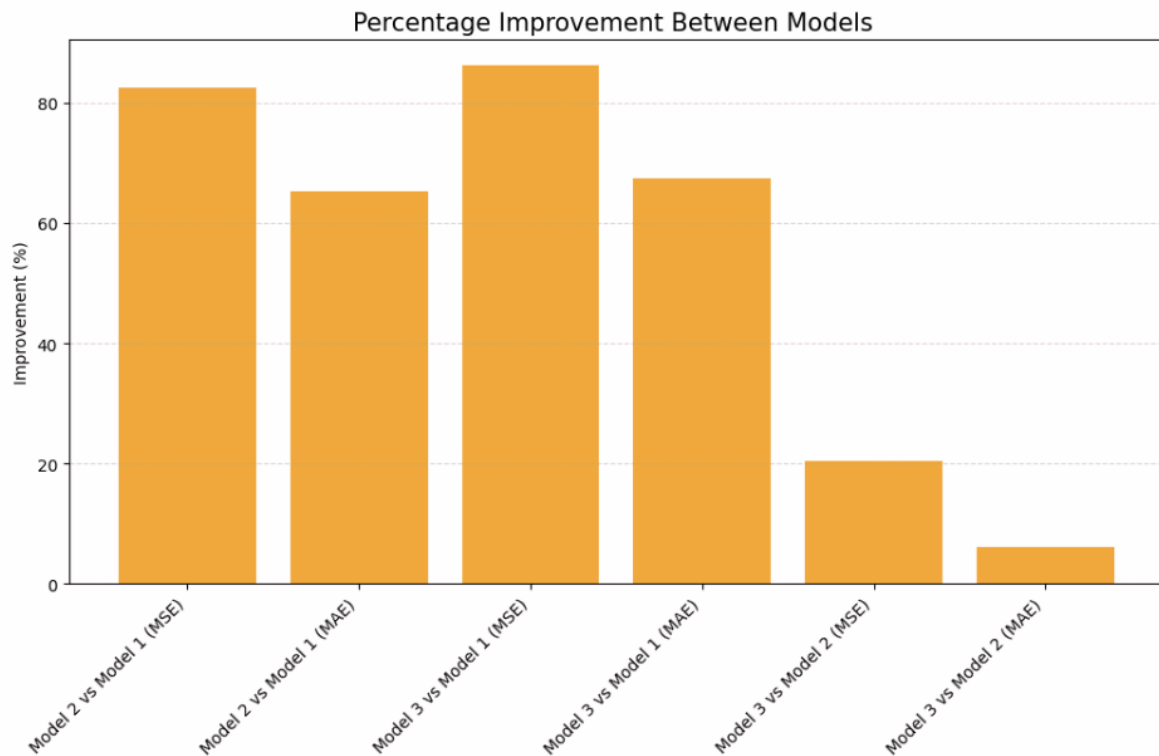
Features Used: G2, G1, Studytime, Failures, Traveltime, Absences, Dalc

MSE: 1.37

MAE: 0.73

This is the most accurate model. It uses only the strongest predictors identified through correlation and feature importance. By removing noisy features, the model becomes more stable and achieves the best performance.





This comparison shows that the quality of features is more important than the quantity.

6. CONCLUSION

This project successfully demonstrates how machine learning, combined with thoughtful preprocessing and feature selection, can be used to predict student performance with high accuracy.

Major observations include:

Model	Features Used	MSE	MAE	Result
Model1	2	9.93	2.26	Low accuracy
Model 2	12	1.72	0.78	Good
Model 3	7	1.37	0.73	Best performance

The results of this project clearly show that G1 and G2 are the strongest indicators of a student's final academic performance. These grades reflect a

student's understanding and consistency throughout the term. They provide strong predictive value for G3. Along with academic history, the study highlighted the impact of behavioral factors like alcohol consumption, absences, and study time. These factors significantly influence learning habits and overall discipline, which shape academic outcomes.

Among the three models developed, the Strong Feature Regression model was the most effective. It achieved the highest accuracy by focusing only on the most relevant features. This supports the idea that choosing meaningful predictors often leads to better results than using a large and unfocused set of variables. At the same time, creating a manual Gradient Descent model improved our understanding of how Linear Regression works, especially how weights adjust to minimize error.

Overall, this study shows how machine learning and data analysis can help improve educational planning and decision-making. By pinpointing the key factors that affect student grades, these predictive models can help teachers intervene sooner, guide students more effectively, and lead to better academic outcomes.

7. FUTURE WORK

To improve the system, several extensions can be implemented:

Model Enhancements

- Use algorithms like Random Forest, XGBoost, and Support Vector Regression.
- Implement Polynomial Regression to capture non-linear relationships.

Deployment

- Build a Streamlit-based web application for real-time grade prediction.

- Integrate the system with school databases for automatic data updates.

Dataset Improvement

- Expand data collection across schools for broader generalization.
- Add psychological, environmental, and motivational features for deeper insights.

Optimization

- Use hyperparameter tuning with GridSearchCV and RandomSearchCV.
- Try regularization methods like Ridge and Lasso Regression.

8. REFERENCES

1. 1. UCI Machine Learning Repository, Portuguese Student Performance Dataset
2. 2. Scikit-learn Documentation, Linear Regression
3. 3. Research Papers on Educational Data Mining
4. 4. Gradient Descent Optimization Techniques
5. 5. Project Dataset and Slides Provided